Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра прикладної математики

ЗВІТ

до лабораторної роботи № 2
на тему: «Web тестування з використанням Selenium»
з дисципліни «Автоматизовані системи тестування програмного продукту»

Варіант 2: сайт XYZ Bank, вхід Customer Login, операція Withdraw

Виконав:
студент групи ПМ-33
Викович Назарій Назарійович

Прийняв:
доц. каф. ПМ, к. т. н.
Любінський Б. Б.

Львів – 2026

1. Мета роботи

Закріпити на практиці автоматизоване тестування Web-застосунків: навчитися керувати браузером засобами Selenium WebDriver, описувати перевірки мовою Gherkin за методологією BDD, організовувати код тестів за шаблоном Page Object, а також знаходити, відтворювати та документувати дефекти застосунку, що тестується.

2. Постановка задачі

Задано демонстраційний банківський Web-застосунок XYZ Bank, розміщений за адресою https://www.globalsqa.com/angularJs-protractor/BankingProject. Відповідно до варіанта 2 необхідно виконати автоматизоване тестування операції зняття коштів (Withdraw), вхід у застосунок здійснювати через Customer Login. Тести мають бути побудовані із застосуванням шаблону Page Objects Pattern та методології BDD із описом сценаріїв мовою Gherkin.

Для виконання роботи потрібно:

1. дослідити застосунок, що тестується: його сторінки, елементи керування та поведінку операції Withdraw;
2. визначити джерела вимог, оскільки письмової специфікації для застосунку не існує;
3. спроєктувати набір тестових сценаріїв, що охоплює коректну роботу операції, граничні значення та некоректні вхідні дані;
4. описати сценарії мовою Gherkin і реалізувати їх прив’язку до коду C#;
5. реалізувати об’єкти сторінок відповідно до шаблону Page Object;
6. забезпечити відтворюваність тестових даних;
7. виконати прогін, проаналізувати результати та задокументувати виявлені дефекти;
8. оформити звіт та розмістити проєкт у репозиторії GitHub.

3. Короткі теоретичні відомості

3.1. Рівень тестування та тестова піраміда

Тестова піраміда описує оптимальний розподіл автоматизованих тестів у проєкті: основу складають модульні тести (60–70 %), середину — інтеграційні (20–30 %), вершину — тести користувацького інтерфейсу, або наскрізні (близько 10 %). Тести UI найповільніші, найкрихкіші та найдорожчі в підтримці, проте лише вони перевіряють систему так, як її бачить користувач.

Тести цієї роботи належать саме до вершини піраміди: Selenium керує справжнім браузером, який відкриває справжній сайт, тому перевіряється

система в цілому. Модульними такі тести бути не можуть за визначенням, оскільки в їх виконанні беруть участь браузер, мережа та зовнішній застосунок.

3.2. Selenium WebDriver

Selenium — набір інструментів для автоматизації дій у Web-браузері. Складові набору: Selenium IDE (запис і відтворення дій без програмування), Selenium WebDriver (програмний інтерфейс керування браузером), Selenium Grid (розподілене виконання тестів) та Selenium Manager. Останній з'явився у версії 4.6 і самостійно визначає версію встановленого браузера та підбирає відповідний драйвер, тому окремо завантажувати chromedriver більше не потрібно.

3.3. Методологія BDD та мова Gherkin

BDD (Behaviour-Driven Development) — підхід, за яким перевірку спершу описують зрозумілою людині мовою як поведінку системи, і лише потім реалізують у коді. Сценарії записують мовою Gherkin у файлах з розширенням .feature. Основні ключові слова: Feature (функціональність), Background (спільна підготовка), Scenario (сценарій), Scenario Outline разом з Examples (шаблон сценарію з таблицею даних), а також Given, When, Then і And.

Структура Given — When — Then відповідає шаблону AAA (Arrange — Act — Assert), відомому з модульного тестування: Given готує стан системи до дії, When виконує саму дію, Then перевіряє результат. Конструкція Scenario Outline з таблицею Examples є прямим аналогом тестів, керованих даними: з одного шаблону утворюється окремий тест на кожен рядок таблиці.

3.4. Reqnroll замість SpecFlow

Завдання передбачає використання фреймворку SpecFlow, який перетворює сценарії Gherkin на тести .NET. Розробку SpecFlow припинено, тому в роботі застосовано Reqnroll — його прямий наступник, створений тим самим автором на основі того самого коду. Мова сценаріїв (Gherkin), атрибути прив'язки ([Binding], [Given], [When], [Then]) та логіка роботи повністю збігаються; змінилася лише назва простору імен: замість TechTalk.SpecFlow використовується Reqnroll. Заміну погоджено з викладачем.

3.5. Шаблон Page Object

Page Object Pattern — шаблон організації коду, за яким кожна сторінка застосунку описується окремим класом. Клас зберігає локатори елементів цієї сторінки та методи дій над нею, а тест звертається лише до цих методів.

Головне правило шаблону: жодного локатора поза класами сторінок. Крок тесту формулюється як «увійти під клієнтом», а не «знайти елемент з ідентифікатором userSelect». Завдяки цьому зміна розмітки сайту вимагає правки одного класу, а не всіх сценаріїв.

3.6. Локатори

Локатор — рядок, що однозначно визначає елемент сторінки. Порядок переваги при виборі локатора: ідентифікатор (id), ім’я (name), CSS-селектор, XPath. Абсолютних шляхів вигляду /html/body/div[2]/form/input слід уникати: вони руйнуються від додавання будь-якого тега. Так само небажані локатори за класами оформлення, оскільки класи описують вигляд, а не призначення елемента.

3.7. Явні та неявні очікування

Сторінка на AngularJS оновлюється асинхронно, тому елемент може з’явитися пізніше за звернення до нього. Неявне очікування (implicit wait) — глобальна вказівка драйверу повторювати пошук елемента протягом заданого часу. Явне очікування (explicit wait, клас WebDriverWait) — очікування конкретної умови у конкретному місці. Змішувати їх не можна: очікування накладаються, і час пошуку стає непередбачуваним. У цій роботі неявне очікування вимкнено (дорівнює нулю), а всі очікування є явними.

Фіксована пауза (Thread.Sleep) не застосовується: вона або марно сповільнює прогін, або спрацьовує зарано на повільній машині, роблячи тест нестабільним.

4. Опис об’єкта тестування

4.1. Застосунок XYZ Bank

XYZ Bank — демонстраційний банківський застосунок, побудований на фреймворку AngularJS 1.4.7. Сервера він не має: усі дані зберігаються у сховищі браузера localStorage. Застосунок надає два входи — Customer Login (кабінет клієнта) та Bank Manager Login (кабінет менеджера). Відповідно до варіанта використовується перший.

Після входу клієнт потрапляє на сторінку рахунку, де відображаються номер рахунку, баланс і валюта, а також доступні три вкладки: Transactions (перелік операцій), Deposit (поповнення) і Withdrawl (зняття коштів). Предметом перевірки за варіантом є остання.

4.2. Операція Withdraw

Форма зняття коштів містить поле для введення суми та кнопку Withdraw. Результат операції застосунок повідомляє текстом над формою. Аналіз вихідного коду застосунку показав, що для цієї операції передбачено рівно два тексти повідомлень, наведені у таблиці 1.

Таблиця 1 — Повідомлення, які застосунок визначає для операції Withdraw

Подія	Файл застосунку	Текст повідомлення
Успішне зняття	`withdrawlController.js`	`Transaction successful`
Сума більша за баланс	`withdrawlController.js`	`Transaction Failed. You can not withdraw amount more than the balance.`
Успішне поповнення	`depositController.js`	`Deposit Successful`

Ці тексти є власним контрактом застосунку перед користувачем, тому вони використані як джерело вимоги. У коді тестів вони винесені константами до класу TestSettings: якщо застосунок колись перефразує повідомлення, правити доведеться один рядок, а не десяток сценаріїв.

5. Підготовка тестових даних

5.1. Спосіб зберігання даних застосунком

Застосунок зберігає клієнтів, рахунки та операції у localStorage браузера. Якщо сховище порожнє, під час завантаження виконується службовий модуль mockDataLoadService.js, який створює початковий набір даних: п’ятьох клієнтів, по три рахунки кожному (у доларах, фунтах і рупіях), а також операції — і лише для першого клієнта на першому рахунку.

5.2. Розрахунок початкового балансу

Модуль додає операції у вкладених циклах: сім місяців по двадцять вісім днів, причому щодня виконується поповнення на 30 і зняття 4. Звідси баланс рахунку 1001 клієнтки Hermoine Granger одразу після скидання стану дорівнює:

$$7 \times 28 \times (30 - 4) = 196 \times 26 = 5096$$

Решта чотирнадцять рахунків мають нульовий баланс. Саме тому за основного клієнта обрано Hermoine Granger: на нульовому балансі неможливо перевірити успішне зняття коштів.

5.3. Скидання стану перед кожним сценарієм

Демонстраційний сайт є спільним для всіх відвідувачів, тому без додаткових заходів баланс був би непередбачуваним. Щоб зробити тести відтворюваними, перед кожним сценарієм виконується крок скидання стану: очищається localStorage і сторінка перезавантажується, після чого застосунок наново створює свій початковий набір даних.

```
public HomePage ResetApplicationState()
{
    ExecuteScript("window.localStorage.clear();");
    Driver.Navigate().Refresh();
    WaitForVisible(MainHeading);
    return this;
}
```

Завдяки цьому кожен сценарій починається з балансу 5096, і в очікуваних результатах можна вказувати конкретні суми. Слід підкреслити: число 5096 є передумовою (Arrange), тобто відомою точкою відліку, а не очікуваним результатом. Самі очікувані суми обчислені з вимоги «зняття зменшує баланс рівно на зняту суму».

6. Структура проєкту тестів

Проєкт побудовано як чотири шари, кожен з яких має рівно одну причину змінитися. Склад файлів наведено у таблиці 2.

Таблиця 2 — Склад проєкту

Файл	Шар	Призначення
`Features/Withdraw.feature`	Сценарії	Вісімнадцять сценаріїв мовою Gherkin, згрупованих за предметом перевірки
`Features/WithdrawReliability.feature`	Сценарії	Перевірка надійності: відсутність необробленого винятку
`StepDefinitions/WithdrawStepDefinitions.cs`	Кроки	П’ятнадцять визначень кроків; усі перевірки (Assert) зосереджено тут
`Pages/BasePage.cs`	Сторінки	Спільне для всіх сторінок: драйвер, явні очікування, клік, введення тексту
`Pages/HomePage.cs`	Сторінки	Стартова сторінка, скидання стану застосунку
`Pages/CustomerLoginPage.cs`	Сторінки	Вибір клієнта зі списку та вхід
`Pages/AccountPage.cs`	Сторінки	Сторінка рахунку, вкладки, форма Withdraw
`Support/TestSettings.cs`	Інфраструктура	Адреса застосунку, клієнт, таймаути, тексти повідомлень
`Support/DriverFactory.cs`	Інфраструктура	Створення й налаштування браузера
`Support/BrowserHooks.cs`	Інфраструктура	Відкриття та закриття браузера, знімок екрана при падінні

Файл	Шар	Призначення
`Support/ConsoleErrorCollector.cs`	Інфраструктура	Читання журналу консолі браузера
`reqnroll.json`	Інфраструктура	Мова Gherkin та культура розбору чисел

Розподіл читається так: Features визначає, що перевіряється; StepDefinitions — як перевіряється; Pages — де шукати елементи на сторінці; Support — усе інше. Жодного локатора поза текою Pages у проєкті немає, що і є виконанням головної вимоги шаблону Page Object.

Перелік використаних пакетів наведено у таблиці 3.

Таблиця 3 — Пакети NuGet

Пакет	Версія	Призначення
`Reqnroll.MsTest`	3.3.4	Розбір Gherkin, генерація тестів, зв’язок з MSTest
`Microsoft.NET.Test.Sdk`	17.11.1	Виявлення проєкту як тестового
`MSTest.TestAdapter`	3.6.4	Виконання тестів у Test Explorer
`MSTest.TestFramework`	3.6.4	Класи перевірок Assert та CollectionAssert
`Selenium.WebDriver`	4.49.0	Керування браузером
`Selenium.Support`	4.49.0	Робота з випадними списками, явні очікування

Цільова платформа — .NET 10. Окремий пакет для генерації коду зі сценаріїв, на відміну від SpecFlow, не потрібен: Reqnroll.MsTest містить генератор у своєму складі.

7. Реалізація

7.1. Життєвий цикл сценарію

Відкриття та закриття браузера винесено у хуки Reqnroll — методи, позначені атрибутами [BeforeScenario] і [AfterScenario]. Це аналог [TestInitialize] та [TestCleanup] з модульного тестування, але на рівні сценарію.

```
[Binding]
public class BrowserHooks
{
    [BeforeScenario]
    public void OpenBrowser()
    {
        _driver = DriverFactory.Create();
        _container.RegisterInstanceAs(_driver);
    }

    [AfterScenario]
    public void CloseBrowser()
    {
        try
        {
```

```
            if (_scenarioContext.TestError != null && _driver != null)
                SaveScreenshot();
        }
        finally
        {
            _driver?.Quit();
            _driver?.Dispose();
        }
    }
}
```

Кожен сценарій отримує власний браузер. Разом зі скиданням стану застосунку це забезпечує незалежність тестів: жоден сценарій не може вплинути на наступний, і порядок виконання не має значення.

Драйвер передається класу кроків через контейнер залежностей Reqnroll: хук реєструє його викликом RegisterInstanceAs, а клас кроків оголошує IWebDriver параметром конструктора. Завдяки цьому клас кроків не створює браузер самостійно, і всі кроки сценарію працюють з одним екземпляром.

Якщо сценарій завершився помилкою, перед закриттям браузера зберігається знімок екрана. Блок finally гарантує закриття браузера в будь-якому разі — інакше після кожного невдалого прогону в системі залишалися б процеси Chrome.

7.2. Об’єкти сторінок і локатори

Локатори взято з вихідних шаблонів застосунку та перевірено на живому сайті. Перелік наведено у таблиці 4.

Таблиця 4 — Локатори елементів

Елемент	Локатор	Обґрунтування
Кнопка Customer Login	`button[ng-click='customer()']`	Класи Bootstrap однакові в обох кнопок входу; атрибут ng-click унікальний і пов’язаний з логікою
Список клієнтів	`id = userSelect`	Наявний унікальний ідентифікатор — найнадійніший варіант
Кнопка Login	`form[name='myForm'] button[type='submit']`	Кнопка має ng-show і до вибору клієнта прихована
Список рахунків	`id = accountSelect`	Унікальний ідентифікатор
Блок даних рахунку	`// div[contains(@class,'center') ][strong]`	Потрібен блок, у якому є нащадок strong; CSS не вміє вибирати елемент за наявністю дитини
Вкладка Withdrawl	`button[ng-click='withdrawl()']`	Прив’язка до логіки застосунку
Поле суми	`input[ng-model='amount']`	Прив’язка до моделі AngularJS
Кнопка Withdraw	`form[ng-submit='withdrawl()'] button[type='submit']`	Уточнено формою, щоб не сплутати з формою поповнення
Повідомлення	`span[ng-show='message']`	Елемент присутній завжди; за порожнього тексту AngularJS додає клас

Елемент	Локатор	Обґрунтування
		ng-hide

Під час перевірки локаторів на живому сайті виявлено три особливості, які довелося врахувати. По-перше, директива ng-options формує значення пунктів списку рахунків у вигляді «number:1001», а не «1001», тому вибір виконується за видимим текстом. По-друге, у шаблоні після номера рахунку стоїть пробіл, отже текст обов’язково обрізається. По-третє, кнопка Login прихована до вибору клієнта, тому порядок дій «спершу вибір, потім клік» є вимогою сторінки, а не стилістичним рішенням.

7.3. Читання балансу

Баланс є основною величиною, проти якої виконується більшість перевірок. У розмітці він розташований у другому з трьох тегів strong усередині блоку даних рахунку:

```
<div ng-hide="noAccount" class="center">
    Account Number : <strong>{{accountNo}} </strong>,
    Balance : <strong>{{amount}}</strong> ,
    Currency : <strong>{{currency}}</strong>
</div>

public decimal GetBalance()
{
    var text = InfoValues()[1].Text.Trim();

    if (!decimal.TryParse(text, NumberStyles.Any,
                          CultureInfo.InvariantCulture, out var balance))
        throw new FormatException(
            $"Не вдалося прочитати баланс: на сторінці «{text}»");

    return balance;
}
```

Тип decimal обрано свідомо: мова йде про грошові суми, а тип double зберігає десяткові дроби наближено. Розбір виконується в інваріантній культурі, оскільки сторінка друкує дробову частину через крапку незалежно від мови операційної системи; у культурі системи, де роздільником є кома, значення прочиталося б неправильно. Метод TryParse у поєднанні з власним текстом винятку дає зрозуміле повідомлення про те, що саме було на сторінці.

Елемент шукається заново при кожному читанні, а не зберігається у полі класу. AngularJS після кожної операції перемальовує блок даних рахунку, і збережене посилання стало б недійсним — Selenium повернув би StaleElementReferenceException. Тому об’єкт сторінки зберігає локатор, тобто правило пошуку, а не результат пошуку.

7.4. Прив’язка кроків

Reqnroll зіставляє рядок сценарію з методом за регулярним виразом в атрибуті. Дужки у виразі визначають параметри методу.

```
[When(@"the customer withdraws (-?[\d.]+)")]
public void WhenTheCustomerWithdraws(decimal amount)
{
    _balanceBeforeOperation = _accountPage.GetBalance();
    _accountPage.Withdraw(amount);
}
```

Один такий метод обслуговує і зняття 100, і зняття −100, і зняття 10.5. Перетворення рядка на тип decimal виконує Reqnroll відповідно до культури, вказаної у файлі reqnroll.json (en-US), — саме тому роздільником дробової частини в сценаріях є крапка.

Важливо розрізняти призначення перевірок у різних блоках. У кроках Given перевіряється лише те, що підготовка відбулася; такі перевірки є сторожовими, і їх повідомлення починаються словами «ПЕРЕДУМОВА НЕ ВИКОНАНА». Оцінка того, що зробила сама операція, виконується виключно у кроках Then. Для початкового балансу передбачено окремий крок, щоб його падіння неможливо було сплутати з перевіркою результату.

8. Джерела очікуваного результату

Кожен тест порівнює фактичний результат з очікуваним, і очікуваний результат має походити з вимоги, а не з фактичної поведінки застосунку. Якби очікування виводили із самого застосунку, тест підтверджував би лише те, що програма робить те, що робить, і не виявив би жодного дефекту; більше того, помилкова поведінка була б зафіксована як норма.

Письмової специфікації для XYZ Bank не існує, оскільки це чужий демонстраційний сайт. Тому джерелами вимоги обрано три групи тверджень, наведені у таблиці 5. Джерело вказано у коментарі над кожною групою сценаріїв у файлі Withdraw.feature.

Таблиця 5 — Джерела вимоги

№	Джерело	Приклад застосування
1	Те, що застосунок заявляє про себе: тексти повідомлень і підписи форми	Звіряння точних текстів у сценарії з тегом contract
2	Поведінка застосунку в аналогічному випадку — вимога послідовності	Для суми 100 застосунок відповідає повідомленням, отже для суми 0 мовчати не має права (дефекти Д1, Д2, Д4, Д6)
3	Загальні вимоги до банківської операції	Баланс не буває від’ємним; зняття зменшує баланс рівно на зняту суму; сума в доларах може містити центи (дефект Д5)

Початковий баланс 5096, хоч і встановлений за вихідним кодом застосунку, не є порушенням цього правила: він використовується як передумова, тобто як відомі вхідні дані, тоді як очікувані результати обчислюються з вимоги.

9. Перелік тестових сценаріїв

Сценарії згруповано за предметом перевірки, а не за очікуваним результатом. Групування за результатом було б помилковим: результат стає відомим лише після прогону, і після виправлення дефекту сценарій довелося б переносити в інший файл. Ознака дефекту — це тег defect, який лише позначає поточний стан.

Два сценарії є шаблонами (Scenario Outline) з таблицями Examples, тому з дев’ятнадцяти сценаріїв утворюються двадцять чотири тести. Повний перелік наведено у таблиці 6.

Таблиця 6 — Тестові сценарії

№	Сценарій	Тестів	Стан
1	Withdrawing part of the balance succeeds	1	пройдено
2	The balance decreases by exactly the withdrawn amount	4	пройдено
3	Consecutive withdrawals are accumulated	1	пройдено
4	A withdrawal affects only the selected account	1	пройдено
5	A withdrawal does not change the account number or the currency	1	пройдено
6	The application reports outcomes with the exact texts it declares	1	пройдено
7	Withdrawing the whole balance is answered and leaves no negative balance	1	пройдено
8	Withdrawing one unit more than the balance is rejected	1	пройдено
9	A withdrawal larger than the balance never changes the balance	3	пройдено
10	Withdrawing from an empty account is rejected	1	пройдено
11	D1 - Withdrawing zero is answered with a message	1	дефект
12	D2 - The amount field is cleared after a zero withdrawal	1	дефект
13	D4 - A negative amount is answered with a message	1	дефект
14	A negative amount does not change the balance	1	пройдено
15	D5 - An amount with cents can be withdrawn	1	дефект
16	The amount field is cleared after a successful withdrawal	1	пройдено
17	The amount field is cleared after a rejected withdrawal	1	пройдено
18	D6 - The error message disappears when another account is selected	1	дефект
19	D3 - Withdrawing zero does not raise an unhandled JavaScript exception	1	дефект

10. Результати виконання

Прогін виконано у середовищі Visual Studio через Test Explorer, тривалість — 2,7 хвилини. Результат: двадцять чотири тести, з них

вісімнадцять пройдено і шість завершилися помилкою. Кожне падіння відповідає окремому виявленому дефекту застосунку.

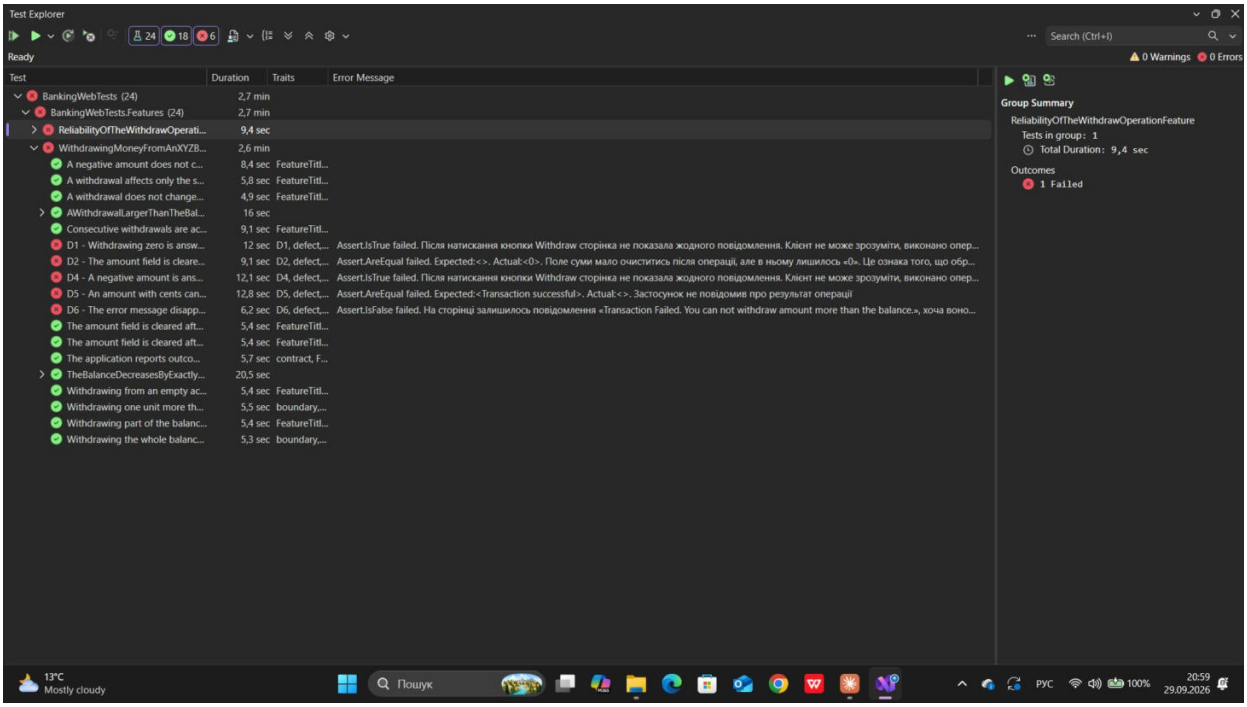


Рисунок 1 — Результати прогону в Test Explorer

Слід підкреслити, що червоний колір шести тестів не є ознакою помилки в тестах. Згідно з визначенням, збій — це відхилення фактичного результату роботи від очікуваного, яке спостерігається при виконанні програми. Червоний тест фіксує саме такий збій, тобто виконує своє основне призначення. Ці сценарії написані так само, як і решта, і з тих самих джерел вимоги; вони не проходять тому, що застосунок цих вимог не виконує. Оскільки застосунок є чужим і виправити його неможливо, тести залишаються червоними — так само, як у реальному проєкті тест лишається червоним, доки розробник не виправить дефект.

11. Виявлені дефекти

Виявлено шість дефектів. Причину чотирьох із них простежено до конкретних рядків вихідного коду застосунку. Перелік наведено у таблиці 7.

Таблиця 7 — Виявлені дефекти

№	Дефект	Опис
Д1	Зняття суми 0 не дає жодного повідомлення	Клієнт натискає кнопку, і сторінка не реагує; неможливо зрозуміти, виконано операцію чи ні
Д2	Поле суми не очищається після зняття 0	У полі лишається введене значення — ознака того, що обробник обірвався, не дійшовши до очищення
Д3	Необроблений виняток JavaScript	TypeError: Cannot read properties of undefined (reading ’success’) у withdrawlController.js, рядок 10
Д4	Від’ємна сума обробляється так	Валідації знака суми немає; поведінка збігається з

№	Дефект	Опис
	само мовчки	випадком нуля
Д5	Суму з копійками зняти неможливо	Поле має type="number" без атрибута step, тому крок дорівнює одиниці, і браузер відхиляє будь-яке дробове значення власною перевіркою
Д6	Повідомлення про помилку не зникає при зміні рахунку	Текст про помилку на рахунку 1001 лишається на екрані після переходу на рахунок 1002

11.1. Спільна причина дефектів Д1–Д4

Аналіз вихідного коду застосунку показав, що дефекти Д1, Д2, Д3 і Д4 мають одну причину. У службовому модулі transaction.service.js змінна результату оголошується, але створюється лише всередині умовних блоків:

```
function transaction(user, account, depAmt, wdAmt, date) {
    var txObj;                                  // оголошено, не створено

    if (depAmt && depAmt > 0) { txObj = {...} }
    if (wdAmt  && wdAmt  > 0) { txObj = {...} }

    return txObj;        // для 0 і від’ємних повертає undefined
}
```

Для суми 0 або від’ємної обидві умови хибні, і функція повертає undefined. Обробник у withdrawlController.js одразу звертається до властивості цього значення:

```
var txObj = Transaction.withdrawl(...);
if (!txObj.success) {            // ← TypeError
    $scope.message = "Transaction Failed...";
} else {
    $scope.message = "Transaction successful";
}
$scope.amount = "";              // ← сюди виконання не доходить
```

Виникає виняток, обробник обривається і не встигає ані показати повідомлення, ані очистити поле суми. AngularJS перехоплює виняток власним механізмом і лише записує його до консолі, тому зовні сторінка виглядає справною. Саме тому для дефекту Д3 знадобилося читання журналу консолі браузера.

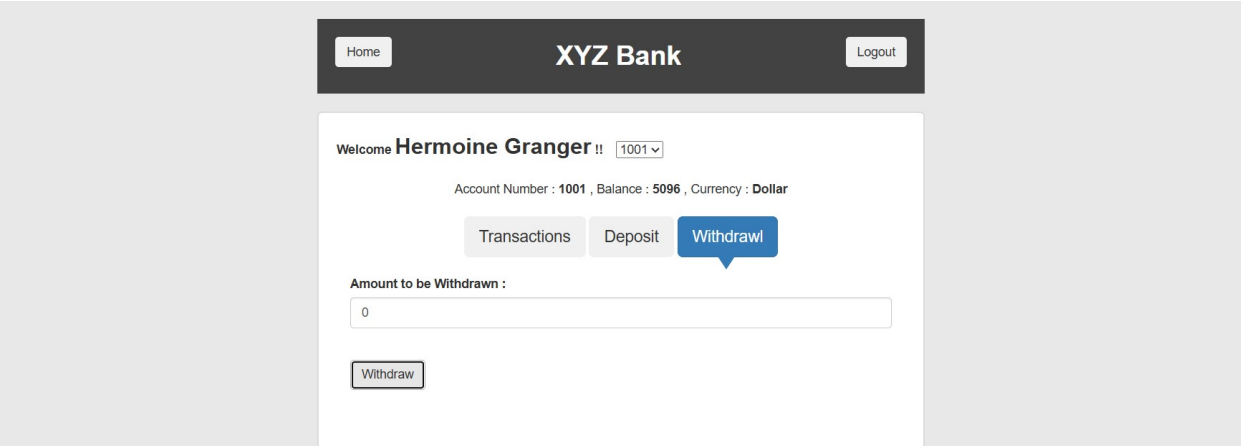


Рисунок 2 — Дефект Д1: після зняття суми 0 повідомлення відсутнє, поле не очищене

11.2. Дефект Д5

Рахунок 1001 ведеться в доларах, отже сума 10,50 є звичайною для зняття. Проте поле введення має тип number без атрибута step, тому крок за замовчуванням дорівнює одиниці, і браузер відхиляє будь-яке дробове значення власною перевіркою, не відправляючи форму. Клієнт не може зняти суму з копійками взагалі.

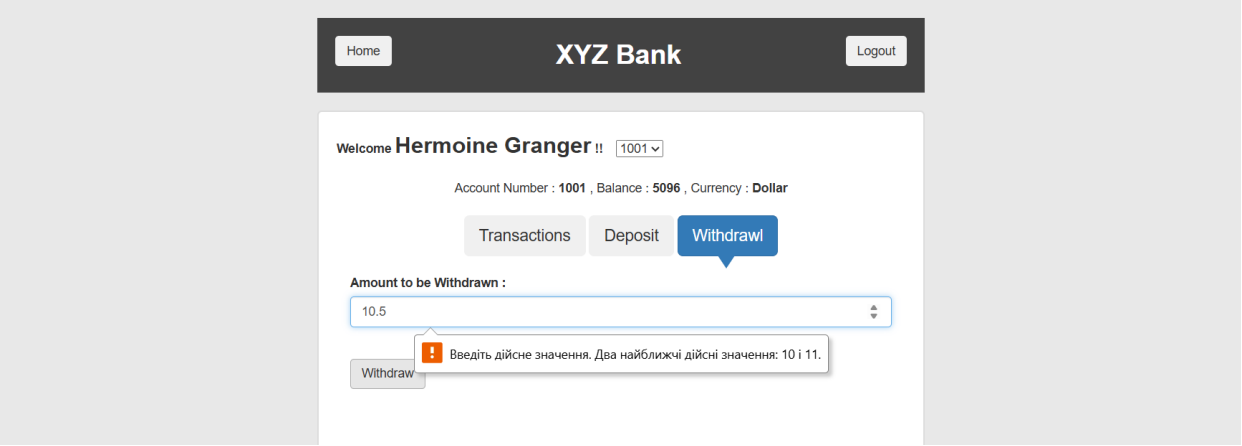


Рисунок 3 — Дефект Д5: браузер блокує відправку форми з дробовою сумою

11.3. Дефект Д6

Після відмови в операції повідомлення лишається на екрані й тоді, коли клієнт перемикається на інший рахунок. У обробнику вибору рахунку немає жодного рядка, який очищав би текст повідомлення, тому користувач бачить помилку, що стосується вже іншого рахунку.

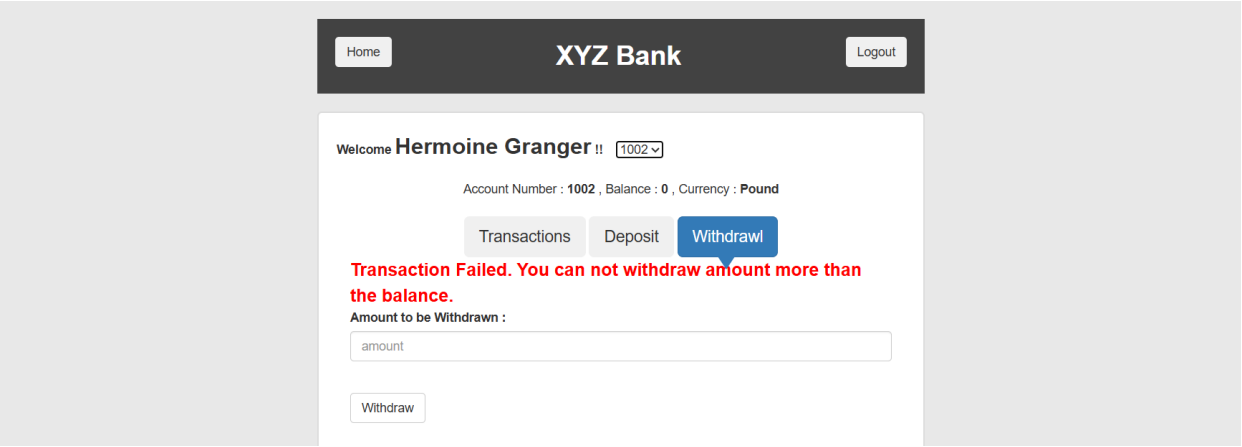


Рисунок 4 — Дефект Д6: повідомлення про помилку лишилося після зміни рахунку

12. Проблеми, виявлені під час прогону, та їх усунення

Аналіз вихідного коду застосунку виявився недостатнім: перший прогін на реальній машині виявив чотири проблеми, дві з яких стосувалися самих тестів і були суттєвими. Перелік наведено у таблиці 8.

Таблиця 8 — Проблеми та їх усунення

Прояв	Причина	Усунення
Тести не запускалися	Проєкт цілив у .NET 8, на машині встановлено лише .NET 10	Цільову платформу змінено на net10.0
Усі 24 тести падали на кроці входу	Перевірка сторінки рахунку виконувалася миттєво, без очікування, а неявні очікування вимкнено навмисно	Додано явне очікування та виведення поточної адреси в текст помилки
Дефект Д3 не виявлявся: тест був зеленим при наявному дефекті	AngularJS захоплює посилання на console.error під час запуску застосунку, тому підміна цієї функції з тесту не спрацьовувала	Перехід на читання журналу консолі браузера засобами WebDriver
Дефект Д5 було сформульовано хибно	Припускалося, що застосунок приймає надто малі суми; насправді браузер блокує відправку форми, і застосунок такої суми не отримує	Сценарій переформульовано на справжній дефект — неможливість зняти суму з копійками

Третя з цих проблем заслуговує окремої уваги. Тест, який залишається зеленим за наявності дефекту, є найнебезпечнішим різновидом помилки в тестовому наборі: він створює хибну впевненість у якості. Виявити його вдалося лише порівнянням результату прогону з поведінкою, відтвореною вручну в браузері.

13. Висновки

У ході виконання лабораторної роботи виконано автоматизоване тестування операції зняття коштів (Withdraw) демонстраційного застосунку XYZ Bank. Побудовано набір з двадцяти чотирьох тестів, описаних

дев’ятнадцятьма сценаріями мовою Gherkin за методологією BDD. Керування браузером виконує Selenium WebDriver, перетворення сценаріїв на тести — фреймворк Reqnroll у зв’язці з MSTest.

Код організовано за шаблоном Page Object: локатори зосереджено виключно в класах сторінок, а сценарії та кроки не містять жодної згадки про розмітку. Завдяки цьому зміна верстки сайту вимагатиме правки одного класу замість перегляду всіх сценаріїв.

Відтворюваність тестових даних забезпечено скиданням сховища localStorage перед кожним сценарієм: застосунок наново створює наперед відомий набір даних, тому кожен сценарій починається з балансу 5096, а очікувані суми є точними значеннями, а не наближеннями.

За результатами прогону вісімнадцять тестів пройдено, шість виявили дефекти застосунку. Причину чотирьох дефектів простежено до конкретного рядка вихідного коду сайту. Окремо слід зазначити, що два дефекти було спершу сформульовано неправильно на підставі самого лише аналізу коду, і виявив це саме реальний прогін — що підтверджує: статичний аналіз не замінює виконання тестів.

Основний методичний висновок роботи полягає в тому, що очікуваний результат тесту має походити з вимоги, а не з фактичної поведінки програми. За відсутності письмової специфікації джерелом вимоги можуть слугувати власний контракт застосунку, вимога послідовності його поведінки та загальні вимоги предметної області. Підганяння очікувань під фактичну поведінку зробило б набір повністю зеленим, але позбавило б його здатності виявляти дефекти.

14. Посилання на репозиторій

Вихідний код проєкту тестів розміщено у репозиторії GitHub:
https://github.com/Juniour-j/Calculator_Exam_CommandProject